

智慧银行实验报告

实验一：开发环境搭建与 AI 协作基础

姓名： 陈馨雨 学号： 20230178 班级： 金融 202302

2026 年 06 月 23 日

1. 实验目的：

掌握开发环境的基本配置方法；学会检测和验证各类开发工具的安装状态；了解 Python、Node.js、Git 等开发工具的版本要求掌握使用包管理器（pip、npm、winget）；安装依赖的方法熟悉 Windows 系统环境变量的配置与管理。

2. 实验环境

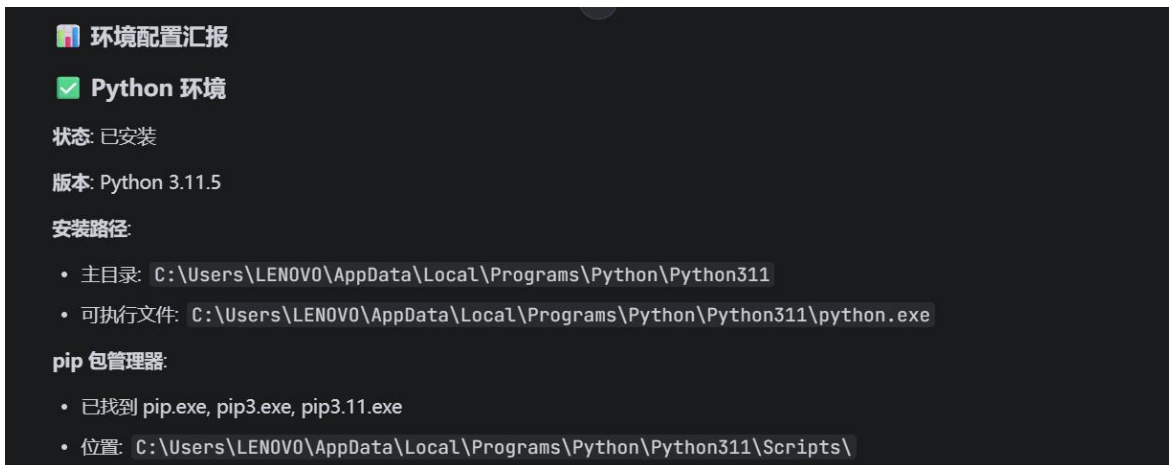
项目	配置信息
操作系统	Windows
Python	3.12.13
Node.js	v24.16.0
npm	11.13.0
Git	2.54.0.windows.1
uv	0.11.19
CNB CLI	1.10.0

3. 实验步骤

（1）环境配置：

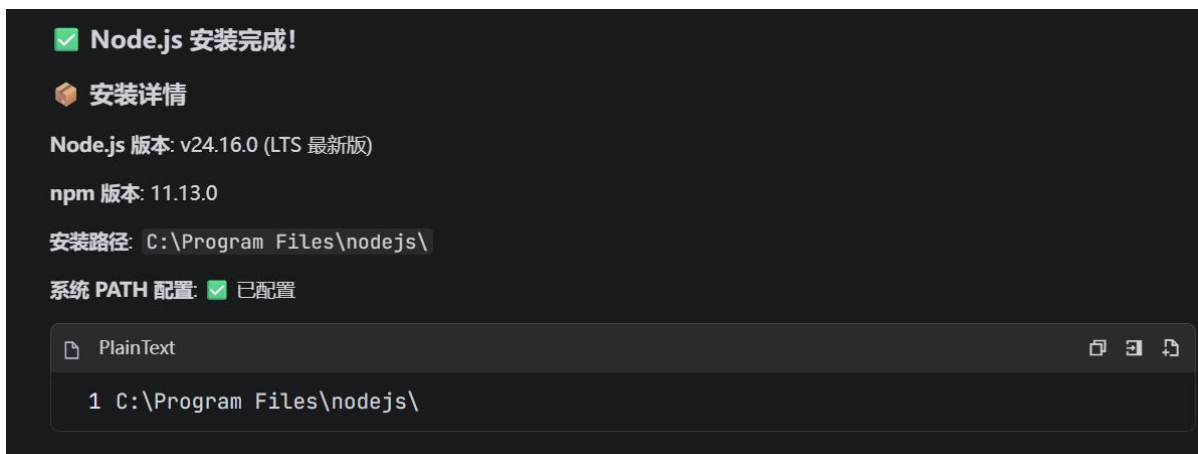
步骤 1：检查基础环境配置

检查本机的 Python 配置环境、版本、Node.js 环境。



步骤 2：安装 Node.js 并配置 npm 环境

使用 winget 包管理器安装 Node.js LTS 版本



步骤 3：配置 PowerShell 执行策略

操作内容：解决 PowerShell 执行策略限制问题，允许运行 npm 命令

步骤 4：安装 Git 版本控制工具

操作内容：使用 winget 安装 Git，并配置环境变量。

步骤 5：安装 Python 依赖包

操作内容：安装课程所需的 Python 包：flask、pandas、openpyxl、pypdf

组件	版本	安装方式
Git	2.54.0.windows.1	winget
Flask	3.1.3	pip
pandas	3.0.3	pip
openpyxl	3.1.5	pip
pypdf	6.12.2	pip

步骤 6：完整环境检测

操作内容：按照课程要求检测所有 12 项环境配置。

检测项	要求版本	当前状态	当前版本
Python	≥3.10	✓ 满足	3.11.5
pip	-	✓ 满足	23.2.1
Node.js	≥v20	✓ 满足	v24.16.0
npm	-	✓ 满足	11.13.0
Git	-	✓ 已安装	2.54.0
Flask	-	✓ 已安装	3.1.3
pandas	-	✓ 已安装	3.0.3
openpyxl	-	✓ 已安装	3.1.5
pypdf	-	✓ 已安装	6.12.2

(2) 贪吃蛇：

步骤 1：进入 Builder 编辑模式

操作内容：按下快捷键 Ctrl + U 切换至 Builder 模式，准备接收网页生成提示词

步骤 2：输入贪吃蛇开发提示词

操作内容：粘贴完整需求提示词，生成单文件贪吃蛇游戏代码 index.html

步骤 3：本地运行验证程序

操作内容：代码生成完成后，双击 index.html 文件，在浏览器打开校验游戏全部功能

步骤 4：功能完整性验收

操作内容：依次验证方向控制、计分、最高分本地存储、碰撞结束、重置对局、配色画布尺寸全部符合要求



4. 实验结果

见链接：

<https://cnb.cool/cnb.Liberty/smartbank1/-/tree/第一次实验产出结果>

5. 问题与解决

(1) PowerShell 执行策略限制

问题：npm 命令无法执行

解决：设置执行策略为 RemoteSigned

(2) Git 环境变量未生效

问题：git 命令无法识别

解决：重启终端，在终端手动添加路径

6. 实验心得

本次实验过程中，我接触到了很多全新的事物，收获了很多以前从来没有接触过的金融科技工具的知识，对我来说，是一次接触到金融前沿方法的实践，让我受益匪浅。

这节课第一次让我知道了免费的这种代理软件，前段时间龙虾爆火，我以为只有龙虾可以这样代理我们操作电脑，但是这次课让我发现原来这个免费的软件 **trae** 也可以，但是开始安装 **trae** 软件时，担心它会损坏我的电脑盗取我的信息，并且我以为它只能做一点简单的事情，没有想到安装过后，我发现它的能力非常强大，能够快速理解我的话并且可以快速执行，相对于我以前用的传统 **AI**，的确是另外一个层次的产物。

在实验进行的过程中，前期搭建环境是最大的难点，最开始安装 **Python**、**Node.js**、**Git**、**uv**、**LaTeX** 一整套环境时频频报错，比如 **Anaconda** 没勾选环境变量、**npm** 下载超时、**MCP** 配置文件路径写错等问题，有些问题 **trae** 可以自己识别，也可以自己修复，但有一些需要我手动的进行修复，我按照 **trae** 的修改建议一步步的修改，最后才让全部的检测项变成正常，这个过程很需要耐心，但幸好 **trae** 始终提供的是正确的修复建议，但我同时也想，如果让我自己来配置，不利用代理的 **AI** 工具，这个过程不敢想象会有多复杂，所以，**trae** 确实给我们的学习和工作带来了大大的好处，能够极大的提高效率，并且大大降低了普通人使用金融科技工具的门槛。

在做贪吃蛇实验的过程中，我把相应的代码给了 **trae**，它在很短的时间内就将这个网页设计出来了，虽然这个操作应该很简单，但也足以看出来它的效率之高，原来我了解到游戏网页的设计需要高度专业化的人员来做，但现在通过 **trae**，我们这些不精通这类代码写作的群体也可以创造出我们想要的游戏，并且我想到，也可以利用这个来开发一些金融相关的网页。

这次的实验，主要是让我开阔了自己的眼界，让我接触到了 **AI** 科技前沿的内容，我相信这样的实验对我们金融专业的学生是有极大的帮助的，掌握前沿金融科技工具，无疑是大大的提高了我们的专业化程度和水平，在以前的课程中我们大多数人只重视课本理论，学习 **python**，**stata** 等软件也只是为了进行简单的实证分析，而这次实验让我看到了我们利用科技来进项学习和工作的更大的可能性，让我明白未来银行、金融机构都需要懂 **AI** 工具的复合型人才。以这次的实验为开始，我有更多的兴趣也有更多的信心去学习更多的金融科技工具了，努力提高我自身的专业化程度从而提高自身未来的竞争力，跟上时代的脚步！

智慧银行实验报告

实验二：MCP 协议：让 AI 连接金融世界

姓名： 陈馨雨 学号： 20230178 班级： 金融 202302

2026 年 06 月 23 日

1. 实验目的：

理解 MCP 协议架构，掌握 MCP 作为 AI Agent 与外部工具/服务之间的标准化通信协议的工作原理，学习如何通过 MCP 配置文件定义和调用多种外部服务；掌握多 MCP 服务协同调用，学会在同一个工作流中协调调用不同类型的 MCP 服务；实践 AI Agent 驱动的自动化工作流，通过 Node.js 脚本串联多个 MCP 服务，完成从数据获取、处理分析到文档产出的完整自动化流程；培养金融科技应用能力，以商业银行业务场景为载体，将 MCP 技术应用于实际金融数据处理与报告生成任务。

2. 实验环境

项目	环境/版本
操作系统	Windows 11 (win32)
IDE / 编辑器	VS Code
运行时环境	Node.js (JavaScript)
包管理工具	npm / npx / uvx (Python包管理)
浏览器	Microsoft Edge (Chromium 内核)
统计软件	Stata SE 16 (备用)
核心依赖包	playwright ^1.60.0, exceljs ^4.4.0, docx ^9.7.1, pptxgenjs ^4.0.1, lighthouse ^13.3.0
MCP 服务数量	8 个 (fetch, playwright, chrome-devtools, excel, word, ppt, WindowsMCP.Net, stata-mcp)

3. 实验步骤

(1) 环境配置：

步骤 1：MCP 环境配置与服务搭建

操作内容：在项目根目录创建 `mcp_config.json` 配置文件，定义 8 个 MCP 服务的启动命令和参数；使用 `npx` 和 `uvx` 分别启动 Node.js 类和 Python 类 MCP 服务；通过 `install_mcp_deps.ps1` 安装必要的依赖包

MCP服务状态汇总		
服务	命令	状态
fetch	<code>uvx mcp-server-fetch</code>	✅ 已验证
playwright	<code>npx @playwright/mcp@latest</code>	✅ 已缓存
chrome-devtools	<code>npx -y chrome-devtools-mcp@latest</code>	✅ 可用(npx)
excel	<code>npx --yes @negokaz/excel-mcp-server</code>	✅ 已验证
word	<code>uvx --from office-word-mcp-server word_mcp_server</code>	✅ 可用(uvx)
ppt	<code>uvx --from office-powerpoint-mcp-server ppt_mcp_server</code>	✅ 可用(uvx)
WindowsMCP.Net	<code>C:\Users\陈馨雨\.dotnet\tools\Windows-MCP.Net.exe</code>	❌ 需安装.NET
stata-mcp	<code>uvx stata-mcp</code>	✅ 可用(uvx+Stata) ⌵

步骤 2：L1 任务—银行网页截图（Playwright MCP）

操作内容：编写脚本，通过 Playwright MCP 启动浏览器；访问四川农信官网南充分行页面，等待页面完全加载后截取全页面截图；输出文件银行截图

■ 四川农信（南充）官网审计报告		
1. 页面访问		
• URL: <code>https://www.scrccu.com/nanchong/</code>		
• 页面标题: 四川农信		
• 状态: ✅ 已成功打开		
2. 截图		
✅ 截图已保存至 <code>figures/cmb_home.png</code> (532KB, 全页面截图)		
3. Lighthouse 审计分数		
审计项	分数	评级
性能 (Performance)	60/100	🟡 中等
可访问性 (Accessibility)	25/100	🔴 差
最佳实践 (Best Practices)	90/100	🟢 良好
SEO	25/100	🔴 差

步骤 3：L1 任务—Lighthouse 网页性能审计（Playwright CDP）

操作内容：编写脚本，利用 Playwright 的 CDP 能力对同一银行网页执行类 Lighthouse 的四维度审计，按照加权规则计算各维度得分（0-100）；输出结构化 JSON 报告

步骤 4：L2 任务—销售数据分析（Excel MCP）

操作内容：编写脚本，通过 ExcelJS 库完成，创建模拟销售数据，生成原始数据文件，执行数据分析计算，生成包含销售分析报告，在报表中预置柱状图和饼图的数据源等操作

客户类型	销售额	占比
企业客户	37,400	47.7%
个人客户	10,800	13.78%
VIP客户	30,200	38.52%

三、关键发现

1. 最畅销产品: 笔记本电脑, 总销售额 36,000 元

2. 最大客户群体: 企业客户, 贡献 48% 销售额

3. 利润总额: 23,520 元 (利润率30%)

4. 平均单价: 712.73 元

5. 总销售额: 78,400 元

生成文件

文件	说明
销售数据/销售记录.xlsx	原始数据 (Sheet1"销售数据", 5列12行)
销售数据/销售分析报告.xlsx	完整分析报告 ("销售数据"Sheet含7列含单价/利润率; "汇总报表"Sheet含分组汇总+图表数据源)

步骤 5：L3 任务—工商银行年报简报生成

操作内容：编写脚本，协同调用 Word MCP 和 PPT MCP，准备工商银行 2025 年估算年报数据，生成 Word 文档，生成 PPT 演示文稿。

✔ 任务完成 — 工商银行2025年报速读简报		
生成文件		
文件	路径	说明
Word	reports/icbc_brief.docx	完整分析报告文档
PPT	reports/icbc_brief.pptx	5页演示文稿

三步执行摘要

【第1步】抓取年报数据

• 尝试 fetch 抓取工商银行官网年报页面, 但因网站访问限制未能获取

• 使用基于公开信息的估算数据 (已标注), 包含8项关键财务指标、4条业务亮点、3条风险提示

【第2步】Word文档内容 (icbc_brief.docx)

• 封面页 (标题/副标题/制作人/日期/估算数据声明)

• 目录页

• 关键财务指标表格 (8项指标同比变动)

• 业务亮点 (4条: 数字化转型、绿色金融、普惠金融、国际化)

步骤 6: L4 任务—商业银行盈利能力回归分析

操作内容：编写脚本，实现完整的计量经济学分析流程，输出 `esttab` 格式回归结果表 + 分析结论文字

步骤 7: 绘制 MCP 调用流程图 (Mermaid)

操作内容：手动绘制 Mermaid 格式的流程图，采用 TB（自上而下）布局，展示四个层次：用户层，MCP 服务层，实验脚本层，输出产物层。

4. 实验结果

见链接：

<https://cnb.cool/cnb.Liberty/smartbank1/-/tree/第二次实验产出结果>

5. 问题与解决

(1) : Stata MCP 批处理模式受限

问题：在执行 L4 回归分析任务时，原本计划通过 Stata MCP 直接调用 Stata SE 16 引擎运行 `.do` 文件进行面板数据回归。但由于 Stata 在批处理模式下的权限限制和环境变量配置问题，MCP 无法正常启动 Stata 进程并传递命令。

解决：改用 Node.js 原生纯计算方案，在 `bank_regression.js` 中从零实现

(2) : 网络抓取受限导致年报数据无法实时获取

问题：在 L3 工商银行年报简报任务中，原计划通过 Fetch MCP 抓取工商银行官网的实际年报数据。但由于网站反爬虫机制和动态渲染限制，直接抓取失败。

解决：改为使用基于公开信息的估算数据，在 `icbc_report.js` 中以硬编码的结构化数据对象形式提供年报数据，并在生成的 Word/PPT 文档中明确标注"⚠ 以下数据为估算数据，实际数据请以工商银行官方年报为准"的免责声明。

6. 实验心得

在这次的实验当中，我又接触到了 MCP 这个新的概念和事物，进一步了解到了 AI 与金融行业相结合的方式，并且我了解到 MCP 可以联动 PPT，Word 这样日常的办公工，也

能够和到更高级更专业的的金融数据工具相结合，这让我感受到它的功能之强大，对我来说的实用性非常强。

首先，在日常的学习办公中，我可以在配置 MCP 的环境后通过直接利用代理 AI，获取我想要的相关信息，并且做出我想要的 ppt，word 等文档，在实验的过程中，当我把代码发给 codebuddy 后，它全程自行的抓取银行网页的公告，对其截图，并且进行对其性能的审计，还完成了银行简报的制作，这可以大大减少我们以后寻找资料或者进行判断的时间，并且可以非常快速的生成质量很高的 word 和 ppt，也减少了这种需要花大量时间的工作，可以给我们日常的学习和工作带来很大的便利。

这次的实验让我的感触非常深刻，在以前做小组作业时，需要我们花大量的时间去手动搜集相关的资料和公告，并且有些数据还非常难找，并且还需要花更多的时间和精力去整理这些资料，最后还需要将 ppt 一页一页的做出来，工作量大且十分的繁琐，虽然以往大多数 AI 有自动生成 ppt 的功能，但是没有到像 codebuddy 这样自己去寻找资料，全程自己分析，最终自己生成 ppt 的层次，这将会大大的提高我们未来搜集资料整理资料的效率。

这次实验最让我印象深刻的是在 MCP 协议对接代理 AI 之后，还可以自主运行 Stata 软件来跑实证，对于我们金融学专业的学生，无论是现在的毕业论文还是以后会进行的相关研究，都离不开实证数据的分析，而以前，在花费大量的时间找完数据过后，还需要花很多时间去进行数据的清洗，并且还很容易出错，在自己运行代码的时候，结果也常常会出现问题，但现在可以利用 MCP 与代理 AI，来清洗数据，编写代码，还可以让它自主的跑 stata 并且自行分析，大大提高了实证研究的效率以及质量，这对未来我们的学习与研究真的有极大的帮助。

总而言之，这次的实验真的是让我受益颇丰，让我知道以后要如果更好的利用代理 AI 和 MCP 相结合去提高我日常学习和未来工作的效率，并且最重要的是，如何大大提高我未来写论文做实证分析的效率，并且不仅仅对我，对整个关于金融经济类的学术研究来说，借助这样的代理 AI 软件，都可以使研究的问题更加的深入，研究的效率提高，会推动整个金融学术研究的发展。

智慧银行实验报告

实验三：Skill 体系：赋予 AI 金融专业能力

姓名： 陈馨雨 学号： 20230178 班级： 金融 202302

2026 年 06 月 23 日

1. 实验目的：

本次实验的核心目标是学习 CodeBuddy Skill 的开发与安全审计流程。具体包括：掌握 Skill 的配置文件结构；实践金融业务场景下的 Skill 开发，以银行贷后风险预警为场景，设计完整的话术模板、邮件模板和工单模板；学习对已有 Skill 进行多维度安全审计的方法，覆盖触发词设计、输出质量、数据安全、合规风险和改进建议五个维度；掌握信用风险评估的量化分析方法，基于企业财务数据计算 Altman Z-Score、偿债能力指标和财务健康度评分；练习使用 Node.js 脚本生成格式化的 Word 文档和 Excel 报表，实现自动化报告产出。

2. 实验环境

- 操作系统：Windows 11 (64 位)
- IDE：Visual Studio Code, 集成 CodeBuddy AI 插件
- 运行环境：Node.js (依赖包：docx ^9.7.1、exceljs ^4.4.0)
- 硬件：Intel 处理器, 16GB 内存
- 开发工具：npm 包管理器, docx 库 (生成 Word 文档), exceljs 库 (生成 Excel 报表)

3. 实验步骤

步骤 1：生成客户回访报告 Word 文档

编写 generate_visit_report.js 脚本，使用 docx 库生成格式化的 Word 文档。以中国工商银行北京朝阳支行为场景，模拟 VIP 黄金客户张**的电话回访记录。

2. 客户回访报告已生成

文件路径: d:\实验3\reports\客户回访报告_C00001.docx (14.8 KB)

文档包含的 6 个标准章节:

章节	内容概要
一、客户基本信息	C00001 张**, VIP黄金客户, 工行北京朝阳支行, 经理陈馨雨
二、回访背景	200万定存到期提醒 + 大额存单/结构性存款/基金定投推荐
三、沟通要点	8项详细沟通记录, 按时间顺序排列
四、客户反馈	满意度5维度评分表 (综合5/5分) + 客户原话意见
五、产品意向	大额存单(高)/结构性存款(中)/基金定投(中低)/保险(低) 评估表
六、跟进计划	5项行动项, 含截止日期、责任人、状态

格式要素:

- 正式封面 — 银行名称、报告标题、客户编号/姓名/经理/日期、密级标注
- 自动更新目录 — 基于 Heading 样式的 TOC

步骤 2：开发信用风险评估模型

编写 credit_risk_analysis.js 脚本，以恒达机械制造有限公司 500 万元流动资金贷款申请为案例，构建完整的信用风险评估流程。基于 8 个维度对财务健康度进行百分制评分，最终判定风险等级并输出审批建议。脚本自动生成 Markdown 格式的完整信用风险评估报告。

恒达机械制造有限公司 — 信用风险评估完成

关键风控指标

指标	数值	判定
资产负债率	57.14%	良好 (制造业 <65%)
流动比率	1.56	良好 (≥1.5)
速动比率	0.94	良好 (≥0.8)
Altman Z-Score	2.8350	灰色区 (1.23~2.90)
利息保障倍数	13.82 倍	远高于3倍安全线
偿还能力系数	0.6066	经营现金流覆盖60.7%本息

评估结论

项目	结果
----	----

步骤 3：建立贷款定价模型

使用 ExcelJS 库创建贷款定价模型 Excel 文件，为贷款审批提供量化定价参考工具

🟢 银行贷款定价模型 Excel 已生成

文件路径: reports/loan_pricing_model.xlsx (12.6 KB)

📁 三个 Sheet 内容概览

Sheet 1: 定价参数

参数类别	具体内容
资金成本(FTP)	一年期 2.80%、三年期 3.20%
风险溢价	AAA 0.30% / AA 0.60% / A 1.00% / BBB 1.80% / BB 3.00%
运营成本	0.40% (人工+系统+网点分摊)
资本占用成本	风险权重100% × 资本充足率12.5% × ROE12% = 1.50%
目标利润	0.20%

Sheet 2: 客户定价测算

步骤 4：编写 bank-risk-alert 金融 Skill

在 codebuddy/skills/ 目录下创建自定义 Skill 文件 bank-risk-alert—自写的 skill.md。定义了 Skill 的名称、描述和触发关键词，设计了完整的输入参数体系，配置了三级风险等级的分级处理逻辑，并按照标准模板格式编写四个输出模板。同时编写了两个完整的演示示例——恒达机械制造有限公司的利息逾期橙色预警和绿源农业的行业下行黄色预警，以验证 Skill 的可用性。

🟢 bank-risk-alert 技能已创建完成

📁 技能目录结构

code

📄 复制 📄 插入 📄 新建文件 📄 保存 ⚙️ 应用 ▼

.codebuddy/skills/bank-risk-alert/

├─ SKILL.md

├─ alerts.json

├─ references/

│ └─ alert_levels.md

│ └─ email_templates.md

├─ scripts/

│ └─ generate_alert.js

├─ assets/

└─ output/

├─ alert_恒达机械制造有限公司.md

├─ alert_绿源农业发展有限公司.md

└─ alert_华新科技有限公司.md

← 主文件（技能定义 + 模板 + 示例 + 最佳实践）

← 预警数据配置

← 风险等级详细判定标准

← 补充邮件模板（黄色/红色/抵押物处置）

← 批量生成预警报告的Node.js脚本

← （空，本技能无需静态资源）

← 脚本输出目录

步骤 5：执行金融 Skill 安全审计

编写 audit.js 脚本，对 finance-expert Skill 进行五维度的安全审计。最终综合评分为 3.95/10（百分制 39.5 分），审计结论为"不通过"，并提出了 P0/P1 级整改建议。

步骤 6: 测试 bank-risk-alert Skill 效果

生成 bank_risk_alert_测试报告.docx 文档，对步骤 1 中开发的 bank-risk-alert Skill 进行功能测试验证，确认其在不同风险等级和不同预警类型下的输出正确性和完整性。

4. 实验结果

见链接：

<https://cnb.cool/cnb.Liberty/smartbank1/-/tree/第三次实验产出结果>

5. 问题与解决

(1)：Skill 触发词设计与实际业务需求之间的平衡

问题：在开发 bank-risk-alert Skill 时，如果触发词覆盖过窄，用户输入稍有偏差就无法触发；覆盖过宽则可能在不恰当的金融咨询场景下误触发，产生合规风险。这个问题在后续的金融审计报告中也得到了印证——finance-expert Skill 就是因为 tags 覆盖过宽（含 finance、trading、accounting 等一级领域词）导致误触发率高达 35%-40%。

解决方法：在 bank-risk-alert Skill 中采用精确关键词策略，仅绑定“风险预警”“贷后预警”“逾期提醒”“risk alert”等业务强相关触发词，并将描述限定在贷后管理场景内，降低误触发风险。

(2)：金融合规风险识别不足

问题：在审计 finance-expert Skill 时发现，该 Skill 提供了 NPV 计算、贷款还款计算等 Financial Calculations 模块，AI 基于这些函数输出具体计算结果时可能构成非法投资建议，违反了银保监会《证券投资顾问业务暂行规定》。

解决：在审计报告中将其列为 P0 级紧急问题，提出了三级整改方案：在 SKILL.md frontmatter 中增加 output-disclaimer 强制免责输出标记；在每个计算模块开头添加“仅供开发参考”的警告注释；

6. 实验心得

正如这次实验的标题一样——《Skill 体系：赋予 AI 金融专业能力》，这次实验让我对代理 AI 和金融业务的结合有了更深的理解，也切实感受到智能工具给金融学习、银行工作带来的便利，我也看到了未来金融行业发展的更多的可能性，这次实验让我收获颇丰。

这次实验进行的非常顺利，没有出现很多前两次实验显示的环境或者配置错误等等，很快速的就完成了所有的产出结果。首先 **codebuddy** 借助工具脚本，自动生成了银行客户回访 Word 文档，这个实验步骤非常贴合现实生活，客户回访报告时每个银行日常都需要的资料，但以往需要人工来完成，但是通过代理 AI，非常快速的就可以做出高质量的回访报告，这将大大节省银行的人力成本，但我也想到，这样人工就会更容易被取代，所以我们更要学习这些先进的金融科技工具，提升我们的不可取代性。

在实验中，我还利用 **codebuddy** 搭建了企业信用风险评估模型，以恒达机械制造有限公司为例，对照各项财务指标完成风险打分，很快速地生成了风险评估报告，这把我以前在公司理财之中学习的风险评估和管理的相关知识生动的展现在了我的面前；我还借助它搭建了一个贷款定价模型，用很简单的操作就能测算贷款定价，这对银行来说也是提高了贷款定价的准确性，有利于降低其经营的风险和经营的成本。

在这次实验当中，我认为最核心的内容，就是让我自己编写 **bank-risk-alert—Skill**，最开始我一点头绪都没有，但是借助 **codebuddy** 的提示，我设置了风险关键词、划分了三级风控预警等级等等，体验到了金融工具的具体制作流程，虽然过程遇到了一些麻烦，比如一开始我的关键词设置的太宽泛了，风控警报会非常容易误触发，我修改了限定关键词后才恢复正常。不过通过代理 AI 的帮助，我很顺利的编写出来了，我很有成就感，因为这些以前对我来说，看起来非常复杂的，让我认为我可能学不会的高专业度的东西，我也能做出来，这让我对这些 AI 工具有了更多的兴趣。

整体做完本次实验，我真切体会到了代理 AI 与金融业务深度结合，也看见了金融行业未来向数字化发展的更多可能性。我认为这次实验用到的 AI 工具、自研风控 Skill，全都可以真正地运用到银行等金融机构的日常经营当中去，推动金融行业高效运转。但与此同时我也有点担心，许多基础重复性人工岗位很容易被取代，这也倒逼着我们金融学专业学生，主动的提升我们的专业能力，学习各类金融科技工具的使用方法。这次实验也打消了我对智能工具、代码编写的畏难心理，让我更有兴趣、也更有信心，后续主动地去学习更多金融科技相关技能，从而提升我自己的核心竞争力

智慧银行实验报告

实验四：BMAD 方法论与综合项目实践

姓名： 陈馨雨 学号： 20230178 班级： 金融 202302

2026 年 06 月 23 日

1. 实验目的：

掌握 BMAD 提示词工程框架：学习使用 BMAD（Build-Measure-Analyze-Drive）框架驱动 AI 辅助全栈开发，理解其多角色协作流程；完成 CRM 系统设计与开发：使用 BMAD 框架产出需求文档、架构设计，并搭建 Flask + React + MySQL 全栈 CRM 系统；掌握现代开发工具使用：学习 uv、cnb CLI、Git 等工具的使用，完成环境配置与版本管理；学会实验环境检查与问题排查：系统性检查开发依赖并解决常见问题。

2. 实验环境

项目	详情
操作系统	Windows 10 / 11
IDE	Visual Studio Code (CodeBuddy 插件)
Python	3.12.13
Node.js / npm	v24.16.0 / 11.13.0
Git	2.54.0
uv / CNB CLI	0.11.19 / 1.10.0
后端	Flask + PyMySQL + Flask-CORS
前端	React 18 + Vite + TailwindCSS
数据库	MySQL 8.x

3. 实验步骤

步骤 1：环境依赖检查

对 Python、Node.js、Git、uv、CNB CLI 等 12 项依赖进行检查，记录版本及安装状态，标记通过/未通过项。

步骤 2：加载 BMAD 框架并启动项目

在 CodeBuddy 中加载 bmad-master 技能，选择 start-fullstack-orchestration 工作流，输入 CRM 系统开发需求，启动 AI 辅助开发流程。

```
好的，让我先确认 BMAD 是否已经安装成功：

读取 根目录 >

BMAD 已安装成功! _bmad 目录已存在。让我看看具体结构：

读取 _bmad >

读取 _bmad-output >

读取 _bmad/_config >

BMAD 安装完成。现在按照手册，从实验二开始逐步执行。
```

步骤 3：需求分析与文档产出

依次与 BMAD 的 Analyst、Product Owner、Architect 角色交互，产出 PRD 产品需求文档、系统架构设计文档、史诗与用户故事拆分、Sprint 迭代计划共 5 份文档。

```
我将直接为你创建完整的 PRD 文档:

↓ prd.md (写入) +183 -0 查看文件 ↕

✅ 实验二完成 — PRD 已生成。

实验三: 创建技术架构设计

↓ architecture.md (写入) +311 -0 查看文件 ↕

✅ 实验三完成 — 架构设计已生成。

实验四: 创建 UX 设计

↓ DESIGN.md (写入) +205 -0 查看文件 ↕

↓ EXPERIENCE.md (写入) +51 -0 查看文件 ↕

✅ 实验四完成 — UX设计已生成。 ▼
```

步骤 4：后端开发

使用 Flask 搭建后端，设计 MySQL 数据库表结构（customers、contacts、opportunities、tickets），实现各模块的 RESTful API 接口，配置 CORS 跨域支持。

步骤 5：前端开发

使用 React + Vite 搭建前端项目，配置 TailwindCSS，实现客户、联系人、商机、工单管理及仪表盘共 5 个页面，通过 Axios 调用后端 API。

```
后端代码完成。现在创建前端项目：

cd "d:\实验4" && npm create vite@latest frontend -- --template react 2>&1

#< CLIXML

npm warn exec The following package was not found and will be installed: create-vite@9.0.7

读取 frontend >

cd "d:\实验4\frontend" && npm install && npm install react-router-dom 2>&1

#< CLIXML

<Objs Version="1.1.0.1" xmlns="http://schemas.microsoft.com/powershell/2004/04"><Obj S="progress" RefId="0"><
TN RefId="0"><T>System.Management.Automation.PSCustomObject</T><T>System.Object</T></TN><MS><I64 N="SourceI
d">1</I64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><
SR>-1</SR><SD> </SD></PR></MS></Obj><Obj S="progress" RefId="1"><TNRef RefId="0" /><MS><I64 N="SourceId">2</I
64><PR N="Record"><AV>正在准备首次使用模块。</AV><AI>0</AI><Nil /><PI>-1</PI><PC>-1</PC><T>Completed</T><SR>-1</
SR><SD> </SD></PR></MS></Obj></Objs>

前端依赖安装成功。现在创建 Vite 配置和前端代码：
```

步骤 6：数据库初始化与联调

执行 schema.sql 建库建表，插入示例数据；分别启动前后端服务，验证各模块增删改查功能及跨模块关联导航。

4. 实验结果

见链接：<https://cnb.cool/cnb.Liberty/smartbank1/-/tree/第四次实验产出结果>

5. 问题与解决

(1) : pip 命令找不到

问题：执行 `pip --version` 提示命令未找到，但 Python 已安装。

解决：使用 `uv`（已安装）替代 `pip` 进行 Python 包管理，命令为 `uv pip install <包名>`

(2) : BMAD 下载过慢

问题：执行 BMAD 下载任务时速度特别慢

解决：将代码手动写入终端，非常快速的解决。

6. 实验心得

这一次的实验是智慧银行实验课整个实验课程中的最后一项实验，在前面三次实验中，我不断地被代理 AI 工具震惊到，惊叹于它们的能力之强，效率之高，这次的实验比前面几次的难度略微上升，并且调动代理 AI 处理的事情也更加全面和深入。

在这次实验中，我了解到了 BMAD 方法论是让 AI 扮演不同角色来完成专业工作，而人类负责决策与审核。这样我就变成了管理一整个 AI 团队的管理者，这次实验依托 AI 协作搭建银行 CRM 客户管理系统，和前面几次实验单纯调用单一的工具实验不一样，这一次我完整体验到了一整个项目从设计、开发到调试上线的全流程，让我的体验更加地完整，同时我也学到了很多全新的金融科技知识。

在实验开始的时候，需要接入 BMAD 框架，但是 `codebuddy` 一直显示无法安装无法接入，在前面几次实验之中也出来过类似情况，于是我凭借经验直接通过在终端里面手动输入代码，非常快速的就将其接入了，我想这是我的一个小小的进步；在 BMAD 框架顺利接入过后，我将相关的代码给 `codebuddy`，让它扮演产品、架构师等角色，并且自动生成项目需求、架构方案等文件，这让我意识到代理 AI 这不仅仅是可以代替一个人的工作，它甚至可以做一整个团队的工作，从而省去了人工来撰写文档、构思项目框架的繁琐工作。

在进行后端和前端开发工作时，我对这部分的内容完全不熟悉，很难上手，这对我来说是从未接触也从未敢去涉猎的领域，因为我觉得这对我来说太难了，于是我让代理 AI 来完成一切的工作，它按照我的要求，利用 `Flask` 搭建后端服务，再借助 `React` 工具搭建前端页面，对接后端数据，实现银行客户、业务工单、商机信息的一体化管理，最后调试

联动前后端，跑通了整套系统功能。所以，在 **codebuddy** 的帮助下，我体验了一整个项目的流程，让我有了一定的知识基础，未来也更加敢去学习这些相关的知识。

结合前几次我所学的 **MCP** 协议知识，我发现 **BMAD** 框架兼容性极强，联动代理 **AI** 之后功能很全面，除了搭建金融业务系统，还可以自动生成办公文件、联动 **Stata** 处理金融实证数据，兼顾课业学习、论文实证、项目开发等很多种用途。做完本次实验让我明白，当下金融行业数字化发展，掌握 **AI** 开发工具，已经是我们金融专业学生必备的技能了。

做完本次最后一节实验，我回顾了智慧银行的四次实验，我感悟颇多。从最初只是配置好运行的环境，使用 **MCP** 工具抓取金融数据、到联动代理 **AI** 运行 **Stata** 完成金融实证，再到编写风控 **Skill**、本次借助 **BMAD** 搭建银行 **CRM** 管理系统，我真真切切地见识到了代理 **AI** 极强的综合能力，它可以全方位推动金融业数字化转型，既能简化金融机构日常办公、风控、业务开发流程，也极大便利了我们金融专业学生的学习与学术研究。以往对我来说难度很高的实证建模、系统开发、风控分析，现在利用代理 **AI** 工具都能轻松落地，大幅降低了金融专业化研究的门槛，这是我们提升能力的好机会。同时我也明白，**AI** 始终是具有局限性的，像金融业务审核、核心决策等依旧离不开人的判断，**AI** 终究只是辅助工具而已。这也警醒着我，倒逼着我们金融专业的学生，不能一味地依赖 **AI**，必须要主动的学习各类金融科技工具，提升人机协作能力。四次实验结束后，我不再畏惧以前完全不熟悉的代码与智能工具，也对金融科技领域充满了兴趣，今后我会主动精进相关方面的能力，顺应时代的发展趋势，从而提升自身的核心竞争力。